COMP3270 2026/27 Coursework 3 Report

Group 27

November 7, 2026

1 Benchmarking Jacobi method in MPI

Figure 1 shows the runtime with the increasing number of MPI processes.

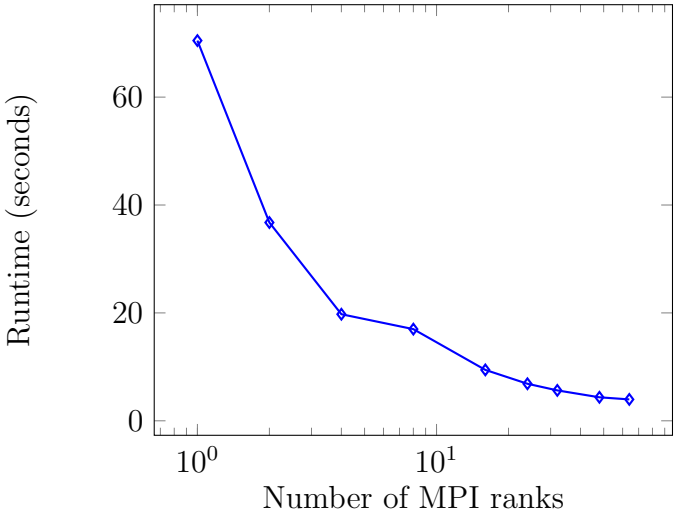


Figure 1: Run-time of the Jacobi method implemented in MPI, across two nodes, up to 32 cores each, with varying number of MPI processes

Accuracy and iteration count: All process counts converged to the same relative residual error of 9.715535×10^{-11} in 181 iterations, so distributing the work did not change the numerical result. A repeat run gave the same error and iteration count, with run times within 1%.

Analysis: The run time falls from 70.48 s on one process to 3.97 s on 64, but the reduction is not linear. The speed-up on 64 processes is only 17.8, with efficiency falling from 96% at $p = 2$ to 28% at $p = 64$.

There is a repeatable step between $p = 4$ and $p = 8$, where the run time only falls from 19.75 s to 16.99 s. In a separate single-node test, the run time stayed at about 23–25 s from 4 to 7 ranks in one NUMA domain, then fell to 14.5 s by 12 ranks once ranks reached a second domain. This is consistent with memory bandwidth saturation, as each rank repeatedly reads a dense matrix too large for cache, although the test ran on different cores from Figure 1.

Rank 0's `generate_system` and `MPI_Scatterv` are serial work inside the timed region, which limits the speed-up by Amdahl's law. `MPI_Allgatherv` and `MPI_Allreduce` also run in all 181 iterations and do not shrink as the calculation per rank does, and above 32 processes some messages must cross the network between nodes.

2 Benchmarking Jacobi method in MPI: distributed generation of the linear system

Figure 2 shows the runtime with the increasing number of MPI processes.

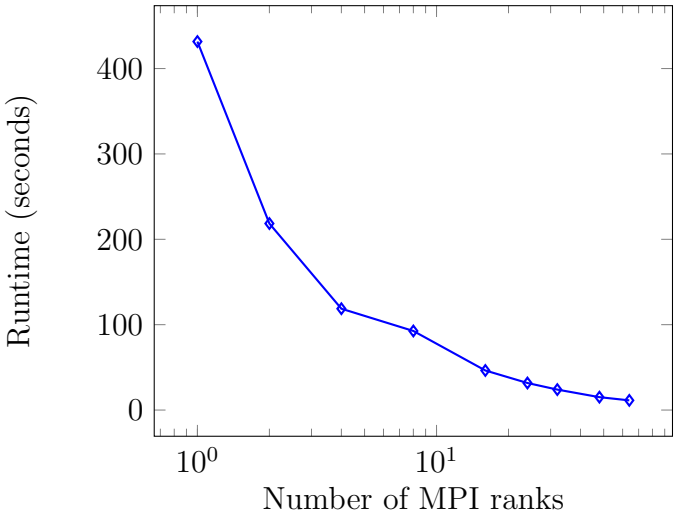


Figure 2: Run-time of the Jacobi method with each rank generating its own rows of the system, $n = 100000$, margin 1.0. Runs with 1–32 processes used one node (150 GB), runs with 48 and 64 processes used two nodes (75 GB each)

Accuracy and iteration count: Every run, from 1 to 64 processes, converged to the same relative residual error of 6.742×10^{-11} in 18 iterations. Because each row is generated from its own random state, `seed + global_i * 2654435761u`, the linear system is identical however the rows are split between ranks. (The brief quotes 20 iterations; the small difference comes from drawing each element of b from the same per-row state as its row of A.)

Analysis: The run time falls from 431.6 s on one process to 11.3 s on 64, a speed-up of 38 (efficiency about 60%). Scaling is almost ideal up to $p = 4$ (speed-up 3.64), drops sharply at $p = 8$ (speed-up 4.66), and then stays at an efficiency of 56–60% up to $p = 64$, with each doubling of processes roughly halving the run time. As in Section 1, this is consistent with memory bandwidth saturation: every entry of A is read twice per iteration (update and residual) for a single multiply-add each, so ranks on the same node compete for memory bandwidth. Going from 32 processes on one node (24.0 s) to 64 on two nodes (11.3 s) gives slightly more than a 2× improvement, because the second node adds its own memory bandwidth. Generation of the system is now parallel, so the remaining parts that do not shrink with p are the `MPI_Allgatherv` of the full x (800 KB) and the synchronising `MPI_Allreduce` in each iteration.

The initial MPI program could not run at this scale. For $n = 100000$, A holds 10^{10} doubles (80 GB), which rank 0 had to generate serially and store in full, on top of its own local rows. In addition, the counts passed to `MPI_Scatterv` (`rows * n`) are `int`s, limited to $2^{31} - 1 \approx 2.1 \times 10^9$ elements, so at low process counts a rank's block of A cannot even be described to MPI. Generating rows locally removes both problems: the measured peak memory was 80.8 GB on one node for the single-node runs and 42.3 GB per node with 64 processes over two nodes. As the course QOS limits each user to 150 GB in total, the provided script (150 GB per node on two nodes) was rejected, so the benchmark was split into the two jobs described in Figure 2.

3 Summary of the group work

Approach and tasks

Ivan took Task 1, converting jacobi_solve.c into jacobi_mpi.c. After going through the original code line by line with the group, he implemented the four required parts:

Distribution: Rank 0 generates the system (A, b) and uses MPI_Scatterv to send each process its rows of A and elements of b. Because n may not divide evenly by the number of processes, the first n mod p ranks get one extra row, so the counts and displacements differ between ranks. Gathering x: In each iteration of jacobi(), MPI_Allgatherv collects every process's piece of the new x, so all ranks have the full vector for the next update. Residual: Each rank computes its part of the residual from its local rows, and MPI_Allreduce combines them into the 2-norm. Every rank gets the same value, so all stop at the same iteration. Timing: Run time is measured with MPI_Wtime().

He also updated the Makefile to use mpicc and made the output print iterations in the same format as the serial solver, as the group agreed, so the results can be compared directly. Frederick took Task 2: building the MPI code on Aire, running the benchmark, and writing the Section 1 analysis. Badad worked on Task 3: Distributed generation of the linear system and wrote the analysis for that section. Before benchmarking, Fredrick checked that the code gave the same error as the serial solver on Aire, using small test runs. The group agreed to print iterations the same way as the serial code, so the counts would be directly comparable. Ivan made the change on GitHub. Max compiled and edited the report.

Problems

The main challenges in developing the parallel Jacobi solver were managing local and global indices, synchronising the solution vector using MPI Allgatherv, and ensuring global convergence using MPI Allreduce. These operations also introduced communication overhead, affecting parallel performance.

mpirun crashed on Aire's login node with an error inside the MPI network library (libpsm2). It ran correctly through SLURM on compute nodes, so the crash appeared to be specific to the login node, and we used batch jobs from then on. The benchmark showed a much smaller gain from 4 to 8 processes than expected. We repeated the run and got almost the same results, so it wasn't a one-off. A follow-up test with process binding reported showed little gain from 4 to 7 ranks in one NUMA domain, then improvement once ranks reached a second domain. This is consistent with memory bandwidth saturation, although we didn't measure bandwidth directly.

Successes

All 9 process counts gave the same error (9.715535e-11) in 181 iterations, so distributing the work didn't change the result. The run time fell from about 70 s on 1 process to about 4 s on 64, a speed-up of about 17.8×. The repeat run agreed with the first to within 1

Meetings

The group met online on 27 October to delegate tasks and choose which project to do.Then, the group met in person on 7 November 2026 in the Edward Boyle Library (planned for 1pm) to work on Tasks 1–3.